APIs

ENTSO-E API

SOURCE:
European Network of Transmission System Operators for Electricity (**ENTSO-E**, Transparency Platform)

Provides Market-Clearing Results (**Electricity Prices**).

The prices for tomorrow become available around **13:50** (not fixed).

Resolution of **15min**

OPEN-METEO API

SOURCE:
OPEN-METEO

Essentially a middle man that collects several weather forecasters and makes them easily accessible.

We choose for model ICON_SEAMLESS from the Deutsche Wetterdienst because of 15min resolution and high spatial accuracy. This model uses their best performing 48h model in combination with another one for extension.

Gets updated every 3 hours

EpexPredictor

SOURCE:
GitHub - b3nO: EpexPredict

A Machine Learning predictor (Gradient Boosting - LightGBM) for Day-Ahead prices.

Can provide forecast of up to 16 days from now.

Uses features like date-time, gas prices, weather, previous prices etc.

Runs as a local server within a Docker Container.

BACKGROUND PROCESSES

ENTSO-E

Activates each day at 13:30 and checks every 30 seconds until new prices arrive.

Writes prices from on start of this day to file (parquet).

Called every 15min and up to 30 hours.

Note

Sometimes the future prices are known up to 36h (right after clearing), sometimes however only up to 10h (right before clearing). We want to be able to make forecasts for up to 24h -> How to forecast DA prices: --> EpexPredictor

Extends the parquet file for up to 30 hours (to build in some margin) for prices that are not in there yet. Note that it does not overwrite real-prices from ENTSO-E as those are the real prices that will be used.

OUTPUT

File that always holds 30h of DA prices (real or predicted).

WEATHER

Checks every 5min if new weather forecast has arrived.

In case of new information, gets written to parquet file.

Saved data:
Irradiance-data (GHI, DHI, DNI)
Temperature
Wind Speed

OUTPUT

File that holds weather prediction for three days.

Historic Data Converter

Activated each day at 00:00:05

Converts all JSONL files to parquet files and add each day before the current one to the historic data file.

OUTPUT

One parquet file with all raw historic data before the current day ~600MB for one year at 1Hz

Server_logger (Development site)

1Hz sampling.

Stored as JSONL files with timestamp of the day

Acknowledgment between Pi and Linux PC (flask)

One-day JSONL: ~80MB file

One-day parquet: ~1.2MB file

OUTPUT

Folder with JSONL files with all data from that day about load and solar inverter

OR

Server_logger (Teleport)

1Hz - 4Hz sampling.

Stored as JSONL files with timestamp of the day.

OUTPUT

Folder with JSONL files with all data from that day about load and solar inverter

Data stored in Datalake (might run as separate process)

Program

PV Forecast

Physical Model

Use stored weather data to create solar forecast according to the specifications of the solar installation.

Residual Model

Machine Learning model to eliminate error between physical model and reality.
Based on Random Forest Model.
(Still quite some margin for improvement)

OUTPUT

Serie of solar forecast in kWh per 15min of equal length to weather forecast file.

Load Forecast

Residual Model

Machine Learning Model
(Gradient Boosting - LightGBM (LIGHT) to predicts the gross (strictly consumption) residential load.

Features include date-time and historical samples.

The horizon steps and resolution can be easily adjusted.

OUTPUT

Serie of 24h load forecast from on the point of calling the function.
E.g. Call the function at 22:00:05,
returns forecast from 22:00:00 until 21:45:00 next day

REQUEST

REQUEST

MPC

Optimiser

Use **Electricity Prices**, **Load forecast** and **Solar forecast**
--> Optimal charging and discharging pattern with a **15min resolution.**

Runs every 15 min again and thus adapts to changes that occur in consumption, weather forecast updates and possibly the updated forecast in DA prices.

In my case of no physical battery:
Pass the next-step-solved SOC

OUTPUT

Serie of 96 (dis)charge setpoints for battery (24hours, 4 steps per hour)

SOC

Link to Real World

Teleport

- Read out data from inverters, batteries and P1 Meter with Flash Messages (±1 Hz)
- Send instructions from MPC to Residential Battery
- Save read-out data to Datalake

DEVICES

Battery

Solar Inverter

DC/AC

P1 Meter

EXTRA INFO:
MPC currently runs on Linux Debian (Lenovo Thinkcentre)